

Checklist

Platform Engineering Planning & Delivery

1. Clarify the Project Before Starting

- ☐ Write a proposal or requirements document before committing to a long-running project.
- ☐ Document the current situation, background, constraints, and key principles.
- ☐ Clearly define the problem being solved.
- ☐ Confirm that the team understands the problem before discussing solutions.
- ☐ Describe the realistic solution options considered.
- ☐ Record the trade-offs, costs, and risks of each option.
- ☐ State the chosen solution and why it was selected.
- ☐ Identify major constraints such as time, staffing, dependencies, and migration effort.
- ☐ Define what “done” looks like.
- ☐ Identify early and medium-term milestones.
- ☐ Define measurable success criteria.
- ☐ Review the proposal with engineering leads, product management, and relevant stakeholders.
- ☐ Resolve major disagreements and obtain buy-in before implementation begins.

2. Turn the Proposal Into an Action Plan

- ☐ Create or begin a technical design document.
- ☐ Assign someone to own the testing strategy.
- ☐ Define baseline acceptance criteria.
- ☐ Identify integration points and external dependencies.
- ☐ Confirm that dependent teams have agreed to participate.
- ☐ Treat customer migration work as a first-class project dependency.
- ☐ Estimate the engineering headcount required.

- ☐ Account for testing, integration, migration, and operational work—not only feature development.
- ☐ Identify likely staffing conflicts with other team priorities.

3. Plan for Adoption

- ☐ Define the product or feature name.
- ☐ Identify potential early adopters.
- ☐ Talk directly with target users before assuming they will adopt the solution.
- ☐ Determine what documentation users will need.
- ☐ Decide whether training or enablement is required.
- ☐ Plan internal communication or “marketing” where useful.
- ☐ Decide how the launch will be announced.
- ☐ Define adoption metrics.
- ☐ Include adoption work in the project plan rather than treating launch as the finish line.

4. Break Long Projects Into Milestones

- ☐ Break large projects into incremental deliverables.
- ☐ Make milestones concrete enough to demonstrate progress.
- ☐ Prefer milestones that deliver customer or business value, not only technical completion.
- ☐ For the first 12 months, consider monthly milestones.
- ☐ For later phases, consider quarterly milestones.
- ☐ Attach measurable outcomes to milestones where possible.
- ☐ Include migration, testing, adoption, and operational readiness in milestone planning.
- ☐ Revisit milestones as unknowns become known.

5. Avoid Common Long-Project Failure Modes

- ☐ Check whether the project scope is expanding beyond the original problem.
- ☐ Remove “nice-to-have” goals that do not materially improve the outcome.
- ☐ Avoid attempting a revolutionary redesign when an incremental improvement would work.
- ☐ Avoid designing a highly complex platform from scratch without a proven simpler version.

- ☐ Make sure the problem is sufficiently clear to write a concrete proposal.
- ☐ Do not build a generic platform intended to satisfy every possible customer.
- ☐ Validate that target users actually need the proposed solution.
- ☐ Watch for project turnover, burnout, and loss of institutional knowledge.
- ☐ Keep documentation current enough that new team members can understand why decisions were made.
- ☐ Stop or reshape projects that have become open-ended slogs.

6. Use Project Management at the Right Time

- ☐ Keep engineering and product leadership directly involved in early planning.
- ☐ Do not assume a project manager can compensate for unclear technical planning.
- ☐ Bring in project-management support when coordination risk becomes substantial.
- ☐ Consider stronger PM involvement when deadlines are firm.
- ☐ Consider stronger PM involvement when there are many tasks or team dependencies.
- ☐ Consider stronger PM involvement when organizational scheduling bureaucracy is a major risk in itself.

7. Build a Bottom-Up Roadmap

- ☐ Include **Features / product roadmap** work.

Include **KTLO — Keep the Lights On** work.

- ☐ Include **Mandates** from executive or organizational leadership.
- ☐ Include **System Improvements**.
- ☐ Treat all four categories as real consumers of engineering capacity.

8. Estimate KTLO Work

- ☐ Estimate on-call and incident-response staffing.
- ☐ Estimate customer or user support work.
- ☐ Estimate remediation work from incidents and postmortems.
- ☐ Review historical data from the previous planning period.
- ☐ Remove unusual one-time events when using history to forecast future KTLO.

- ☐ Aim for KTLO to consume **no more than roughly 40% of team capacity**.
- ☐ If KTLO exceeds that level, prioritize work that reduces the operational burden.

9. Account for Mandates

- ☐ List executive or organization-wide mandates separately from normal roadmap work.
- ☐ Include required platform and infrastructure migrations.
- ☐ Include cloud/provider or geographic expansion initiatives.
- ☐ Include acquisition or integration work.
- ☐ Include compliance and security requirements.
- ☐ Include strategic company-wide initiatives.
- ☐ Estimate the actual engineering cost of every mandate.
- ☐ Surface trade-offs when mandates would displace higher-value work.
- ☐ Escalate early when the mandate load exceeds the team's realistic capacity.
- ☐ Work with leadership to remove, delay, or reduce lower-value mandates where possible.

10. Plan System Improvements

Reliability & Operability

- ☐ Identify manual operational work that can be automated.
- ☐ Improve testing to prevent recurring defects from reaching production.
- ☐ Evaluate load testing, fuzz testing, or flaky-test reduction where appropriate.
- ☐ Improve deployment and release-engineering capabilities.
- ☐ Improve observability and monitoring.
- ☐ Reduce unnecessary differences between environments, APIs, configurations, or versions.
- ☐ Address system-level reliability weaknesses.
- ☐ Review postmortem action items for recurring improvement opportunities.
- ☐ Review customer incidents to identify platform guardrails that could prevent future failures.

Efficiency & Performance

- ☐ Identify the systems with the highest cost or the greatest potential for performance improvement.

- ☐ Assign clear ownership for FinOps or cost-efficiency work.
- ☐ Tag cloud and infrastructure resources for ownership.
- ☐ Create useful spend reports for engineering and finance.
- ☐ Review reservation and commitment strategies.
- ☐ Right-size infrastructure and eliminate unused resources.
- ☐ Negotiate vendor discounts where appropriate.
- ☐ Improve forecasting of future infrastructure costs.
- ☐ Give system engineers time to investigate and implement performance improvements.

Security & Compliance

- ☐ Identify high-value security improvements.
- ☐ Include security and compliance work in normal planning rather than waiting for incidents.
- ☐ Prioritize improvements based on probability and impact.
- ☐ Include architectural security changes in larger redesigns where required.
- ☐ Include incremental security improvements alongside reliability and feature work.

11. Rank and Merge the Work

- ☐ Maintain separate ranked lists for reliability, efficiency/performance, and security/compliance.
- ☐ Compare projects within each category before comparing across categories.
- ☐ Combine the product roadmap, KTLO, mandates, and system-improvement lists into one realistic capacity plan.
- ☐ Make trade-offs explicit.
- ☐ Avoid planning as though every request can be delivered.
- ☐ Preserve local team details before rolling plans up to higher organizational levels.
- ☐ Avoid turning the planning process into a headcount competition between teams.

12. Apply Planning Heuristics

- ☐ Conduct a deep planning exercise approximately once per year.
- ☐ Refresh the plan more lightly during the other quarters.

- ☐ Keep individual system-improvement projects to roughly **three developer-months or less** when possible.
- ☐ Reframe larger improvements as part of a broader project instead of letting them become endless maintenance efforts.
- ☐ Consider the **70/20/10 model** for non-KTLO work:
 - ☐ ~70% core initiatives and incremental improvements.
 - ☐ ~20% adjacent innovation or re-architecture.
 - ☐ ~10% transformational innovation or entirely new platforms.
- ☐ Treat 70/20/10 as a discussion tool—not a guaranteed budget entitlement.
- ☐ Leave capacity for important unplanned work.

13. Be Careful With Innersourcing

- ☐ Do not assume other teams will routinely contribute code to the platform they consume.
- ☐ Remember that the platform team remains responsible for maintaining contributed code.
- ☐ Establish contribution rules, review standards, ownership, and support expectations.
- ☐ Avoid using innersourcing as a substitute for customer prioritization discussions.
- ☐ Use contribution models selectively rather than as the platform's default operating model.
- ☐ If external contributions become frequent, investigate whether the platform itself is missing important capabilities or ownership.

14. Track Delivery With Biweekly Wins & Challenges

- ☐ Collect updates approximately every two weeks.
- ☐ Ask teams for a small number of meaningful **Wins**.
- ☐ Ask teams for a small number of meaningful **Challenges**.
- ☐ Roll updates upward through the management hierarchy.
- ☐ Select only the most important items for broader audiences.
- ☐ Rewrite technical details so the intended audience can understand it quickly.
- ☐ Focus on outcomes and impact rather than ticket counts or story points.

For Each Win or Challenge

- ☐ Start with a short, bold summary.
- ☐ Describe the **Situation**: What was happening?

- ☐ Describe the **Action**: What did the team do?
- ☐ Describe the **Result**: What changed?
- ☐ Use numbers or metrics when they improve understanding.
- ☐ Remove unnecessary technical detail.
- ☐ Explain why the update matters to customers, stakeholders, or the business.

15. Include the Right Types of Updates

- ☐ Major project milestones.
- ☐ Delays or delivery risks.
- ☐ Important staffing or organizational changes.
- ☐ Significant incidents or unusually strong operational periods.
- ☐ Changes in engineering or business metrics.
- ☐ Performance improvements.
- ☐ Cost savings.
- ☐ Adoption milestones.
- ☐ Customer satisfaction changes.
- ☐ Important customer feedback.

16. Treat Challenges as Useful Information

- ☐ Share challenges that require management visibility.
- ☐ Surface recurring collaboration problems.
- ☐ Call out any dependencies blocking delivery.
- ☐ Use recurring challenges as evidence for escalation or process improvement.
- ☐ Sharing externally relevant challenges builds trust or enables help.
- ☐ Avoid publicly surprising or blaming a partner team.
- ☐ Discuss cross-team issues with the relevant team before broadly publishing them.

17. Operationalize the Wins & Challenges Process

- ☐ Define exactly who must submit updates.
- ☐ Establish a consistent submission deadline.
- ☐ Provide a standard template.
- ☐ Provide a central place to submit or store updates.
- ☐ Make engineering managers responsible for the final quality of their teams' updates.

- ☐ Allow product managers and other team members to help draft them.
- ☐ Have skip-level managers review and select the most important items.
- ☐ Make important highlights visible to senior leadership.
- ☐ Share notable achievements with peers and stakeholders.
- ☐ Periodically review whether the process is producing useful information.
- ☐ Adjust the format or cadence when it becomes low-value bureaucracy.

18. Final Planning Review

- ☐ Is the problem clear?
- ☐ Is the proposed solution justified?
- ☐ Are the scope and requirements realistic?
- ☐ Are dependencies visible?
- ☐ Is migration work included?
- ☐ Is adoption planned?
- ☐ Are testing and acceptance criteria defined?
- ☐ Are milestones incremental and measurable?
- ☐ Is staffing realistic?
- ☐ Is KTLO explicitly accounted for?
- ☐ Are mandates explicitly accounted for?
- ☐ Are reliability, efficiency, and security improvements represented?
- ☐ Have you intentionally prioritized what **will not** be done?
- ☐ Is there slack for unexpected work?
- ☐ Can stakeholders understand what the team is delivering and why it matters?
- ☐ Is there a regular mechanism for communicating both wins and challenges?